A More Details on Data Generation

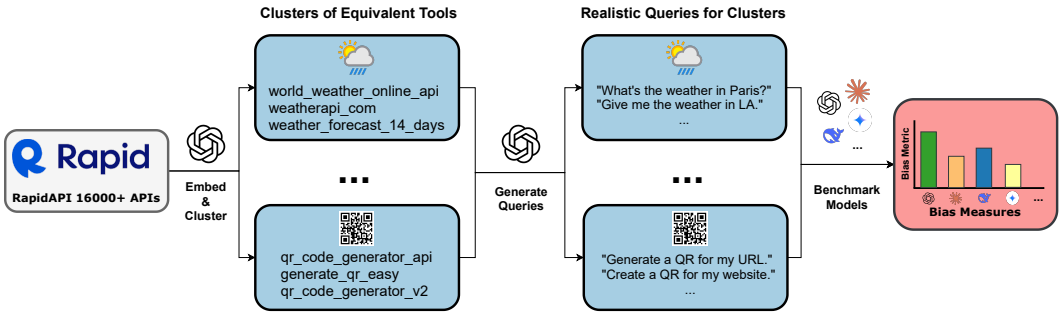

Figure 7: An overview of our clustering and query generation pipeline.

We build on the "tool-usage" evaluation pipeline introduced by Qin et al. (2024), hereafter referred to as ToolLLM, owing to its wide adoption and extensibility. At its core, ToolLLM assembles a large catalog of real-world APIs scraped from RapidAPI spanning 49 functional categories (RapidAPI, 2025b). For each API, ToolLLM provides a JSON file containing the API's human-readable name, detailed description, and full parameter schema. In this work, we leverage exactly that API repository but restrict our attention to the stages in which ToolLLM selects among a short list of retrieved candidates. Note that in ToolLLM, a set of closely-related APIs is called a 'tool'. For example, a geocoding tool could offer both forward geocoding and reverse geocoding APIs[3].

We assemble our benchmark in two stages: (1) clustering APIs into functionally equivalent groups, and (2) generating realistic user queries for each group (see Figure 7).

Algorithm 1 Generation of Functionally-Equivalent API Clusters

Require: API id to metadata map π,
Precomputed embeddings E,
List of "general" APIs $G = \{(\text{tool, tool_desc, api_name, api_desc})\}$,
neighbor count K,
max outlier loops R

$\mathcal{C} \leftarrow \emptyset$
for all $(\text{tool, tool_desc, api_name, api_desc}) \in G$ **do**
  Construct query text $q =$ "tool:tool_desc | api_name:api_desc"
  Embed q with ADA: $\mathbf{v}_q \leftarrow \text{Embed}(q)$
  Compute cosine similarities: $s_i \leftarrow \cos(\mathbf{v}_q, E_i) \quad \forall i$
  Select top–K unique tools with largest similarity and store in set TOP_K
  candidate $\leftarrow \{\pi[i] \mid i \in \text{TOP}_K\}$
  for $r \leftarrow 1$ **to** R **do**
    Prompt GPT-4 to detect outliers: outliers $\leftarrow \text{DetectOutliers}(\text{candidate})$
    if outliers $= \emptyset$ **then break**
    end if
    Remove outliers from candidate: candidate $=$ candidate $\setminus$ outliers
  end for
  if $|\text{candidate}| > 3$ **then**
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{\text{candidate}\}$
  end if
end for
return $\mathcal{C}$

[3]Geocoding is the process of converting between human-readable addresses and geographic coordinates: "forward" geocoding maps an address (e.g., "1600 Amphitheatre Parkway") to its latitude/longitude, while "reverse" geocoding maps a given coordinate pair back to a structured postal address.

```
You are a prompt-writing assistant. I will give you a set of API
    endpoints (tool name + description, endpoint name + description,
    and potentially the required parameters) that all perform the same
     underlying task. Please generate exactly {n} distinct, natural-
    language user queries that could be satisfied by ALL of these
    endpoints. **Include realistic sample values** for any required
    parameters (e.g. use "https://example.com" for a URL, or "Hello
    World" for a text field). Return them as a JSON array of strings,
    with no extra commentary.

User:
Here are the endpoints:
- Tool: WeatherNow - Provides current weather information
  Endpoint: Current Weather - Returns temperature, humidity,
  and conditions for a given location.
  Required parameters:
    * city (string) - name of the city
    * country (string) - ISO country code
- ...
```

Figure 8: Example prompt used for query generation. The model outputs n natural-language queries that all listed endpoints can satisfy.

API clustering. We begin by embedding every endpoint's metadata (tool name, API name, descriptions, etc.) into a shared vector space using a pre-trained text encoder (OpenAI's `text-embedding-ada-002` model). We then curate a small set of "seed" APIs whose descriptions span a number of "general" tasks, such as text translation or weather forecasting. For each seed, we retrieve its top-K nearest neighbors in embedding space to form a candidate cluster. To ensure true functional equivalence, we iteratively prompt GPT-4 to flag any outlier endpoints that cannot perform the same task as the rest; flagged APIs are removed and the check repeats for up to a pre-defined number of rounds. Any cluster that stabilizes with more than three members is retained. See Algorithm 1 for an overview of our clustering approach. Lastly, we manually inspect and refine these clusters, yielding 10 high-quality groups of five APIs each.

Query generation. For each cluster, we prompt GPT-4 (see Figure 8) to produce natural-language queries that all members can satisfy. In batches of ten, the model generates candidate queries until we collect 100 unique queries per cluster, filtering out duplicates. In cases where freeform generation exhibits provider-specific bias (e.g., mentioning a particular vendor's feature), we switch to a template-filling workflow: we design a small set of generic templates with placeholders (e.g. "Get the latest news headlines for {country} about {topic}."), and ask GPT-4 to instantiate each template multiple times with realistic sample values.

Final curation. All 1,000 generated queries are then reviewed by hand to remove any that inadvertently favor a single provider or rely on specialized parameters. The resulting dataset consists of 10 clusters with 5 APIs each, and 100 balanced, provider-agnostic queries for each cluster.

Running each model over these prompts yields empirical selection distributions over APIs and list positions, from which we compute our total-variation-based bias metrics δ_{API}, δ_{pos}, and δ_{model}. This rigorously grounded benchmark enables precise measurement and comparison of tool-selection bias across models and settings.

B Attribute-Level Analysis Feature Table

See Table 3 for the list of features used in the analysis of Section 4.3.

C More Details on the Biased CPT Experiment

We test whether pre-training data can cause tool-selection bias by doing biased continued pre-training (CPT) on a single model. That is, we do additional next-token training on raw text using

Table 3: API-level predictor features.

Feature	Description
avg_similarity_tool_desc	Mean text similarity between cluster queries and the tool's description.
avg_similarity_api_desc	Mean text similarity between cluster queries and each API's description.
age_days	Days since the API was first published.
desc_name_length_sum	Total character count of the API's name plus description.
num_params	Number of required and optional parameters.
flesch_reading_ease	Flesch reading-ease score of the combined descriptions.
positive_word_count	Count of positive or promotional words (e.g. "efficient," "robust").

$\sim$3.5M tokens deliberately saturated with one endpoint's metadata (its name, description, and parameter info). After this exposure, we re-run the selection tasks and measure shifts in that endpoint's selection share.

To generate the biased corpus, we synthesize long-form prose with an external LLM (Gemini 2.5 Flash). We prompt it to produce a single document of roughly 1.1–1.3k words written in a randomly sampled style (e.g., blog note, Q&A memo, release note, how-to guide, troubleshooting checklist). The prompt requires frequent natural mentions of the target API name, the exact or faithfully paraphrased tool description, and the exact or paraphrased API description; it also requests inclusion of the endpoint path in about 60% of documents and parameter metadata in about 50%. This pipeline yields a large, stylistically varied corpus that is nevertheless saturated with the same endpoint's metadata.

We then run CPT on the same base model used elsewhere (i.e., Qwen3-8B) with parameter-efficient adapters (LoRA) attached (see Hu et al. (2022) for more details on LoRA). We keep tokenizer unchanged.

We evaluate pre/post CPT selection distributions on the original cluster, prompts, and inference settings. The primary outcome is the shift in the target API's selection share. We also measure spillover: changes in the selection shares of non-target APIs within the cluster. If biased CPT reliably increases the target API's selection share, this is evidence that a portion of tool-selection bias originates from pre-training exposure.

D Details on the Implementation and Evaluation of the Mitigation Method

After showing the existence and possible causes of bias, we seek to mitigate it. We pursue a simple approach based on the following insight: Models often know which APIs can solve a task but can possibly exhibit biased choices among interchangeable endpoints. We decouple capability recognition from final selection via a lightweight debiasing module.

The debiasing module consists of a lightweight LLM (Qwen3 14B in our case) prompted to output the subset of APIs from the given candidate list that can solve the task given in the query. This way, we get a subset selector that outputs an array of the APIs that can complete the task. The system prompt constrains the output to an exact list with no prose. From the returned set S, we pick one API uniformly at random. This API then replaces the original API list and is used for the rest of the tool-usage pipeline.

If this approach is successful, each API in S has an expected selection share of $1/|S|$, eliminating position/API favoritism at the choice stage. If the selector's true positive/negative rates are high enough, the overall selection distribution approaches uniform even when original models were skewed. This, following our definition, means the tool selection stage becomes unbiased by design.